Speaker Script

An Automated Framework for Dataset Quality Assessment and Data Leakage Detection in Machine Learning

25-minute defense — matches `presentation.pptx` (29 slides)
Texto hablado en **inglés** ů acotaciones en *cursiva gris* en español

Antes de empezar — chuleta de números (si dudas, di el número redondo): **20** checks / **5** dimensiones (el **29** es el *checklist* del benchmark, no lo mezcles) · **12** datasets (6+6) · **325** tests · **4/4** escenarios de leakage vs. 0–1/4 · accuracy **1.000** → **0.952** ($\Delta = -0.048$) · `boat` $\mathcal{L}$ = **0.74** (ablation: cae a 0.662 con esquema correlation-heavy) · `name` Cramér's V = **0.999** · Titanic **80.6/B** · umbral warning **0.7** / error **0.9** · pesos LRS **0.35/0.35/0.30** · pesos readiness **0.25/0.35/0.25/0.15**.

Ritmo: ≈110–115 palabras/min. Si a mitad (slide 16) vas por encima de 13 min, recorta slides 11 y 18 a una frase cada una.

Part 1 — Motivation & Objectives (≈5:40)

Slide 1 — Title 0:20 acum. 0:20

De pie, contacto visual, sonríe. No leas el título entero.

Good morning. My name is Jaime Cano Moraño, and today I will present my master's thesis: an automated framework for dataset quality assessment and data leakage detection in machine learning. This work was developed at the Illinois Institute of Technology and is presented here at ETSIT-UPM.

Slide 2 — Agenda 0:40 acum. 1:00

Señala las cinco secciones con la mano, no las leas palabra por palabra.

In the next twenty-five minutes I will cover five parts. First, why data leakage is a problem worth automating. Second, the system I built and the methodology behind it — this is the core of the talk. Third, the experimental results on twelve datasets. Fourth, a quantitative benchmark against three established tools. And finally, a short live demo, conclusions, and future work.

Slide 3 — Divider 01 0:10 acum. 1:10

Pasa rápido; una sola frase mientras cambia.

Let us start with the motivation.

Slide 4 — Models Fail Because of Data, Not Algorithms 1:30 acum. 2:40

La caja azul de la derecha es la definición: apóyate en ella al decirla.

In machine learning practice, most of our effort goes into models — but most production failures actually trace back to the data: missing values, duplicates, class imbalance, distribution drift. And among all data problems, one stands out as the most insidious: data leakage. The definition is on the right: leakage happens when information about the target reaches the training features in

a way that will not exist at prediction time. What makes it dangerous is that it is completely silent. Every other data problem makes your model look worse. Leakage makes it look *better* — you get inflated validation metrics that collapse the moment the model faces real data. This has been documented in medicine, in finance, and in large-scale reproducibility studies. So the core question of this thesis is: can dataset quality assessment and leakage detection be automated, quantified, and made actionable in a single tool?

Slide 5 — A Motivating Example 1:15 acum. 3:55

Pausa dramática tras decir "one point zero, zero, zero". Señala la flecha.

Let me make this concrete. I trained a logistic regression classifier on a dataset containing one engineered leaky feature. Validation accuracy: one point zero zero zero. Perfect. Any of us would be suspicious of a perfect score — but in a large feature table, with a less extreme leak, this goes unnoticed. After removing the leaky features, the honest accuracy is 0.952. That delta of minus 0.048 is fake performance — the gap between a reproducible result and a broken one. A careful human reviewer might catch this case. But at scale, nobody reviews every feature of every dataset. The framework I will show you detects this automatically — and quantifies the damage.

Slide 6 — The Gap in Existing Tools 1:00 acum. 4:55

Recorre las tres tarjetas de izquierda a derecha.

Of course, data quality tooling already exists. ydata-profiling generates excellent descriptive reports. Great Expectations validates data against hand-written rules. Deepchecks runs train-test validation suites. They are mature and widely used — but when I tested them on four constructed leakage scenarios, they detected at most one out of four, and that single detection required manually authoring a rule. None of them produces an interpretable readiness verdict or code-level fixes. So my framework is positioned as a complement: it fills the leakage-detection and actionability gap, rather than replacing general-purpose profiling.

Slide 7 — Objectives and Contributions 1:15 acum. 6:10

Una frase por tarjeta; los KPI de abajo solo señálalos.

This translates into four contributions. One: a unified, automated framework — twenty deterministic checks across five dimensions, plus impact analysis, accessible from a command line, a Python SDK, and a web application. Two: a unified Leakage Risk Score that combines three statistical signals into one flaggable number per feature. Three: semantic leakage analysis using a large language model, for leaks that statistics cannot see. And four: a quantitative benchmark against the three tools I just mentioned. Everything is validated on twelve datasets and covered by 325 unit tests, and the code is open source.

Part 2 — System & Methodology (≈8:10)

Slide 8 — Divider 02 0:10 acum. 6:20

Transición.

Let me now open the system up and show you how it works.

Slide 9 — System Architecture 1:15 acum. 7:35

Recorre el diagrama de arriba a abajo con el puntero.

The pipeline needs exactly two inputs: a YAML configuration file and a CSV dataset. After loading, the data flows through five analysis dimensions in parallel: six quality checks, five leakage checks, three feature-analysis checks, four statistical sufficiency checks, and two drift checks — twenty in total. Two additional stages complete the picture: impact analysis, which retrains models with and without suspicious features, and an optional LLM-based semantic module. Everything converges into a set of code-level recommendations and a single readiness score, exported as an HTML report and a JSON file.

Slide 10 — Three Interfaces, One Pipeline 0:45 acum. 8:20

Señala el código: subraya que son cinco líneas.

The same pipeline is exposed through three interfaces: a CLI for batch runs and CI/CD, a Python SDK for notebooks — as you can see, a full analysis is about five lines of code — and a Streamlit web application for non-programmers, which I will demo at the end. There is also a plugin API, so teams can register their own domain-specific checks.

Slide 11 — The 20-Check Catalog 0:45 acum. 9:05

No leas la tabla. Una frase por idea. Recortable si vas mal de tiempo.

This is the complete catalog — I will not read it. The key design decision is that every check returns the same structured result: pass or fail, a severity, and the affected columns. That uniformity is what lets failed checks map automatically to twenty recommendation handlers that emit ready-to-use pandas code. The leakage row is where the novelty lives, so let us zoom into its fifth element: the unified risk score.

Slide 12 — Unified Leakage Risk Score L(f) 1:30 acum. 10:35

Este es EL slide técnico. Ve despacio. Señala cada término de la ecuación.

Here is the first main contribution. The problem with single statistics is that each one can be fooled: correlation misses non-linear leaks, mutual information alone is noisy, and both can be diluted by missing values. So for every feature I combine three normalized signals. Rho is the association with the target — Pearson correlation for numeric features, Cramér's V for categorical ones. I-tilde is mutual information, normalized by the maximum across features. And pi is what I call performance inflation: train a model on this single feature and measure how close it gets to perfect accuracy — a direct behavioral test of leakage. The weighted combination, 0.35, 0.35, 0.30, gives a score between zero and one. At 0.7 the framework raises a warning; at 0.9, an error.

Slide 13 — Are the Weights Arbitrary? — Ablation 1:00 acum. 11:35

Anticipa aquí la pregunta típica del tribunal. Señala la fila roja.

A fair question is whether those weights are arbitrary, so I ran a sensitivity analysis on real leaky features from Titanic. The feature *boat* — the lifeboat number — is a true leak, but its raw correlation is diluted by missing values. Under the default weights it scores 0.74 and is correctly flagged. Under a correlation-heavy scheme it drops to 0.662 — below the threshold, a false negative. Every

scheme that gives reasonable weight to mutual information or performance inflation catches it. So the defaults are defensible, not decorative — and they remain fully configurable.

Slide 14 — Semantic Leakage Analysis with an LLM 1:00 acum. 12:35

Ejemplo discharge_date: cuéntalo como una mini-historia.

The second contribution addresses leaks that statistics cannot see at all. Imagine an ICU mortality dataset with a feature called *discharge_date*. Statistically it may look innocent — but semantically it is obvious: you only know the discharge date after the outcome. No statistical test reads feature names. So the framework sends feature names, sample values, and a dataset description to GPT-4o-mini, which returns a risk level, a leakage type — temporal, proxy, post-hoc or indirect — and a natural-language rationale for each feature. The module is optional, disabled by default, and degrades gracefully if no credentials are present.

Slide 15 — Semantic Module: Evaluation and an Honest Caveat 0:45 acum. 13:20

Di el caveat TÚ, con seguridad. Es un punto fuerte, no una debilidad.

To evaluate it, I built a manually labelled thirty-feature benchmark, where the module reaches a precision of 1.0 and a recall of 0.93. But I want to be fully transparent: these figures validate the evaluation harness using a deterministic mock analyzer, because Azure credentials were unavailable at evaluation time — they are not a measurement of the live model. This is stated explicitly in the thesis, and a live evaluation is the top-priority item of future work.

Slide 16 — From 20 Checks to One Number 0:45 acum. 14:05

Chequeo de tiempo: deberías ir por ≈13–14 min aquí.

Finally, all results aggregate into a readiness score from zero to one hundred. Each dimension starts at one hundred and loses fifteen points per error and five per warning; the dimensions are then combined with these weights. Leakage carries the largest weight, 0.35, deliberately: it is the only failure mode that silently inflates results instead of visibly degrading them. The score maps to grades A through F, and every weight and penalty is configurable in the YAML file.

Part 3 — Experimental Results (≈4:35)

Slide 17 — Divider 03 0:10 acum. 14:15

Transición.

Does it work? Let us look at the evidence.

Slide 18 — Experimental Setup: 12 Datasets 0:45 acum. 15:00

Izquierda sintéticos, derecha reales. Frase clave: planted vs. surprises.

I evaluated the framework on twelve datasets. Six are synthetic, with defects I planted deliberately — a clean control, a dirty one, and four leakage scenarios of increasing subtlety. Six are real-world datasets from OpenML and UCI, from Titanic to the 48,000-row Adult Census. The logic is

simple: the synthetic sets verify the framework detects what we planted; the real sets verify it finds surprises in the wild.

Slide 19 — Readiness Scores Discriminate as Designed 1:00 acum. 16:00

Señala las dos barras naranjas.

And it discriminates exactly as designed. The clean control scores 98.8, grade A. The dirty dataset — full of quality noise but with no leakage — stays in grade A at 91.2. Only the two datasets with confirmed target leakage drop to grade B: the synthetic leaky dataset at 71.6, and, interestingly, Titanic at 80.6. The four additional UCI datasets all score in the nineties — with Wine Quality pulled down to 88.6 by fifteen percent duplicate rows. Quality noise costs points; leakage costs grades.

Slide 20 — Case Study: Titanic 1:15 acum. 17:15

La anécdota estrella de la defensa. Cuéntala con calma y algo de humor.

Why does Titanic — the most famous teaching dataset in the world — get a B? The framework found two real leaks. First, *name*: with 1,300 unique values it behaves as a row identifier, and its Cramér's V with survival is 0.999 — flagged as an error. Second, and more interesting, *boat*: the lifeboat number. It scores 0.74 on the unified risk score. Think about what that feature means: you were assigned a lifeboat *because* you survived. It literally encodes the target. Thousands of tutorials train models with this column included — and it takes the framework thirty seconds to prove it is textbook post-hoc leakage. Neither leak is caught by any of the three baseline tools.

Slide 21 — Quantifying the Damage: Impact Analysis 0:45 acum. 18:00

Cierra el círculo con el ejemplo del principio.

The impact analysis module closes the loop on my opening example. It retrains the same cross-validated model with and without the flagged features. On the leaky dataset: baseline 1.000, cleaned 0.952. That measured gap is direct, quantified evidence of leakage — not just a statistical suspicion — and the recommendations then emit the exact pandas code to drop the offending columns.

Part 4 — Benchmark (≈2:10)

Slide 22 — Divider 04 0:10 acum. 18:10

Transición.

How does this compare against the tools practitioners already use?

Slide 23 — Feature Coverage vs. Three Established Tools 0:50 acum. 19:00

Sé escrupuloso con el framing: checklist ≠ pipeline; menciona fairness.

I built a 29-item comparison checklist — note this is a cross-tool measurement instrument, more granular than my own 20-check pipeline; the two numbers measure different things. My framework satisfies all 29 items; the closest alternative, Deepchecks, satisfies 11. One fairness note: these

baselines were designed for profiling and validation, not leakage — so this chart measures scope, not quality, and the full checklist is published with the benchmark code so anyone can scrutinize it.

Slide 24 — Leakage Detection — the Decisive Result 0:50 acum. 19:50

Esta tabla es el resultado decisivo: dilo explícitamente.

The decisive result is this one. Four constructed leakage scenarios, every tool with its default configuration. My framework detects all four automatically. ydata-profiling and Deepchecks detect none. Great Expectations detects one — the ID column — and only because I manually authored a rule for it. This is precisely the gap the thesis set out to fill.

Part 5 — Demo & Conclusions (≈5:00)

Slide 25 — Divider 05 0:05 acum. 19:55

Transición rápida; abre ya la pestaña del navegador con Streamlit.

Let me show you the framework in action.

Slide 26 — Live Demo: Streamlit 3:00 acum. 22:55

DEMO EN VIVO. Ten la app ya lanzada y titanic.csv en el escritorio. Guion de respaldo si falla: enseña el vídeo grabado o el informe HTML y di: "In the interest of time, let me show you a pre-recorded run."

[Durante la demo, narra:] I upload titanic.csv and select *survived* as the target… the pipeline runs the twenty checks plus impact analysis live — this takes a few seconds… Here is the readiness summary: 80.6, grade B, with the per-dimension breakdown. In the leakage tab, you can see *name* flagged as an error and *boat* with a risk score of 0.74, each with an explanation and a suggested fix — and here is the exact pandas line to drop the column. Finally, the full HTML report can be downloaded and shared with the team.

Slide 27 — Conclusions 1:00 acum. 23:55

Una frase por tarjeta, con energía de cierre.

To conclude. First: dataset quality assessment and leakage detection can be automated end to end — one config file plus one CSV gives you a scored, explained, and actionable verdict. Second: the unified risk score detects leaks that no single statistic catches, with ablation-backed weights — four out of four benchmark scenarios versus at most one for existing tools. Third: LLMs extend detection to semantic leaks that are invisible to statistics. And fourth: the framework found real, previously unflagged issues in classic public datasets. On the engineering side: seventeen development phases, fourteen modules, and 325 tests, all passing.

Slide 28 — Future Work 0:40 acum. 24:35

Rápido; el punto 1 conecta con el caveat del slide 15.

Four directions ahead. The immediate priority is the live LLM evaluation — the harness is ready, only credentials are missing. Then: broadening the leakage taxonomy to group and preprocessing

leakage; CI/CD integration, using the readiness score as a merge gate — the SDK already makes this a few lines of code; and learning flagging thresholds from data instead of fixing them.

Slide 29 — Thank you **0:20** **acum. 24:55**

Pausa, sonríe, no salgas corriendo del slide.

The code, the datasets, and the full benchmark are available on GitHub. Thank you very much for your attention — I will be happy to take your questions.

Annex — Preguntas probables del tribunal (con respuesta)

£Por qué esos pesos en $\mathcal{L}(f)$ (0.35/0.35/0.30)?
(EN) They are near-uniform by design — no single signal dominates. The ablation in the thesis shows the flagging decision is stable across reasonable schemes, and that the one clearly bad idea is over-weighting correlation: it reproduces a false negative on Titanic's *boat* (0.662, below the 0.7 threshold). All weights are configurable in the YAML.

£La evaluación del módulo LLM es real?
(EN) No — and the thesis says so explicitly. The P/R/F1 figures validate the evaluation harness with a deterministic mock, because Azure credentials were unavailable. The architecture and parsing are fully tested (33 unit tests). A live evaluation is the first item of future work; the harness runs it with a one-flag change.

£Por qué el umbral 0.95 en target_leakage y 0.7/0.9 en LRS?
(EN) They are conservative defaults chosen to keep false positives low on the clean control (which scores 98.8 with no leakage flags), and they are configurable. Learning calibrated, dataset-size-aware thresholds is listed as future work.

£Escalabilidad? £Coste computacional?
(EN) The heaviest components are mutual information and the single-feature models in $\pi(f)$. The largest dataset evaluated is Adult Census, 48,842 rows, which runs the full pipeline in well under a minute on a laptop. Runtime figures are in the benchmark chapter.

£Por qué GPT-4o-mini y no un modelo local?
(EN) It was the best cost/quality point available through the institutional Azure deployment, and the module only sends feature names, a handful of sample values, and a description — not the dataset. The interface is provider-agnostic: any chat-completion endpoint can be plugged in, including local models.

£No es injusto el benchmark 29/29 contra herramientas de otro propósito?
(EN) That is exactly why the thesis includes a fairness note: the checklist measures *scope*, not quality, and it is published with the benchmark code. The claim is not that ml-framework is better at profiling — it is that leakage detection is absent elsewhere, which the 4/4 vs. 0–1/4 table shows directly.

£Qué pasa con falsos positivos (features legítimamente predictivas)?
(EN) A genuinely predictive feature can score high — that is why flags are warnings with explanations, not automatic deletions. The impact analysis helps disambiguate: a legitimate feature removed causes a real performance drop at deployment too, whereas a leak's performance was never real. Final judgement stays with the practitioner; the framework makes the suspicion visible and quantified.

£Diferencia entre 20, 21, 23 y 29 checks?
(EN) 20 deterministic pipeline checks; +1 optional semantic check (21); a full run reports 22–23 results because impact analysis adds one per configured model; 29 is the separate cross-tool comparison checklist. The thesis uses 20 for the pipeline and 29 only in the benchmark chapter.

Heart Disease: £qué pasó con el target?
(EN) While reconciling numbers across documents I found a genuine data bug: a digit-extracting regex collapsed the OpenML labels into a single class. I fixed the download script, re-ran the entire pipeline on all datasets, and replaced every affected number — the thesis reports only the corrected, reproducible results (96.8/A).